An AI System for Transforming Scattered Web Data into an Interactive, Queryable Knowledge Graph

Mohammed Huzaifah*, Syed Abdul Muqeet Mujeeb†, Niyazuddin Mohammed‡, Mir Danish Ali§
Department of Computer Science (Artificial Intelligence and Machine Learning)
Lords Institute of Engineering and Technology, Hyderabad, Telangana, India
Email: *huzaif027@gmail.com, †a.muqeetmujeeb@gmail.com, ‡niyazuddin607@gmail.com, §mir.danish@gmail.com

***Abstract*—The exponential growth of online information has created major challenges for individuals attempting to organize and retrieve knowledge scattered across web sources. This paper presents MindCanvas, an artificial intelligence-driven system that transforms unstructured, fragmented data into a coherent, interactive knowledge graph supporting semantic search and natural-language querying. The system leverages large language models (LLMs), high-dimensional vector embeddings, and retrieval-augmented generation (RAG) to automatically summarize, categorize, and link information from diverse web sources. Its architecture combines FastAPI-based backend services, a Supabase vector database using the pgvector extension, and a React-based visualization interface that enables intuitive exploration of personal or domain-specific knowledge spaces. Evaluations on a 1 GB web dataset (2,500 pages) demonstrate strong performance: topic extraction precision of 0.84 and recall of 0.79, semantic retrieval MRR@5 of 0.74, and graph metrics (clustering coefficient 0.36, modularity 0.52) that facilitate exploratory learning. User studies with 15 participants show 87% improvement in information recall, 93% success in discovering new connections, and 73% reduction in redundant searching.**

I. Introduction

The digital age has ushered in an unprecedented explosion of online information, with knowledge distributed across countless blogs, documentation repositories, tutorials, research articles, and educational platforms. This fragmentation presents significant challenges for knowledge workers, students, and researchers who must navigate this vast information landscape effectively. Empirical studies indicate that knowledge workers spend a substantial portion of their time—often exceeding 20%—searching for information and attempting to relocate previously viewed content [1]. A common pattern emerges where individuals repeatedly visit similar pages without recognizing prior exposure to the same topics, resulting in inefficient learning processes, poor knowledge retention, and wasted cognitive effort.

Traditional approaches to personal knowledge management, such as manual bookmarking, note-taking applications, and browser history, fail to address the fundamental challenge of establishing meaningful connections between disparate pieces of information. These tools lack the semantic understanding necessary to automatically organize content based on conceptual relationships, making it difficult for users to discover patterns, identify knowledge gaps, or synthesize information from multiple sources effectively.

To address these limitations, we propose MindCanvas, an artificial intelligence-based system that constructs an interactive, queryable knowledge graph from heterogeneous web data. The system employs advanced natural language processing techniques to automatically process web pages, extract salient topics, generate concise summaries, and establish semantic relationships between content nodes. By leveraging 1,536-dimensional vector embeddings and computing cosine similarity between content representations, the system identifies related information and constructs a graph structure where nodes represent processed content and edges represent semantic relationships based on shared topics or similarity thresholds (empirically set at $\theta = 0.7$).

A distinguishing feature of MindCanvas is its conversational interface powered by retrieval-augmented generation (RAG). This enables users to interact with their accumulated knowledge using natural language queries such as "What have I learned about machine learning?" or "Show me the connections between my readings on databases and web development." The RAG-based chatbot retrieves relevant context from the knowledge graph and generates coherent, grounded responses that include provenance information linking back to source materials.

The primary contributions of this work include:

- **Modular Processing Pipeline:** A comprehensive architecture for transforming unstructured browsing data into an indexed, semantic knowledge graph with automated ingestion, processing, and relationship extraction capabilities.
- **Hybrid LLM Integration:** Seamless integration of both commercial (GPT-5) and open-source (Llama 4) large language models for content analysis, combined with state-of-the-art vector embeddings for semantic search and RAG-based conversational interaction.
- **Empirical Validation:** Rigorous experimental evaluation including an ablation study comparing commercial and open LLMs, comprehensive performance metrics, and a user study demonstrating practical benefits for knowledge management tasks.
- **Privacy-Aware Design:** An architecture that supports local-first privacy configurations with options for on-device processing, along with reproducibility artifacts to facilitate community adoption.

II. Related Work

The challenge of organizing and retrieving personal knowledge has motivated research across multiple domains, including personal information management, knowledge graph construction, semantic search systems, and conversational AI.

A. Personal Knowledge Management Tools

Traditional note-taking and knowledge management applications such as Roam Research [2] and Obsidian [3] have gained popularity for their support of bidirectional linking and graph-based visualization of notes. These tools enable users to manually create connections between concepts, forming a network of interlinked notes that can be explored and navigated. However, their effectiveness depends critically on manual effort: users must explicitly create links, assign tags, and organize content. This manual approach presents significant scalability challenges and limits semantic depth, as the systems lack automated understanding of content relationships. Furthermore, these tools primarily operate on user-created content rather than automatically processing external web sources.

B. Web-Scale Knowledge Graphs

Large-scale knowledge graphs such as Google's Knowledge Graph [1] demonstrate the power of structured, semantically-rich information representations for organizing world knowledge. These systems employ curated ontologies, entity recognition, and relationship extraction to create comprehensive knowledge bases covering millions of entities and their interrelationships. However, web-scale knowledge graphs are designed for general public knowledge and are not personalized to individual users' browsing patterns, learning journeys, or domain-specific interests.

C. Embedding-Based Retrieval Systems

Recent advances in dense vector representations have revolutionized information retrieval. Dense Passage Retrieval (DPR) [4] and similar techniques employ neural encoders to map text into high-dimensional vector spaces where semantic similarity corresponds to geometric proximity. This enables more nuanced retrieval compared to traditional keyword-based methods. Retrieval-augmented generation (RAG) architectures [5] further enhance these capabilities by combining retrieval systems with large language models to generate contextually grounded responses.

Vector databases with approximate nearest neighbor search capabilities, such as pgvector for PostgreSQL [6], have made semantic retrieval computationally feasible at scale. Frameworks like LangChain [5] provide composable abstractions for building RAG pipelines, simplifying the development of retrieval-augmented applications.

D. Positioning of MindCanvas

MindCanvas distinguishes itself through several key innovations: (1) automated processing without requiring explicit user annotation, (2) personalized knowledge graphs tailored to individual information consumption patterns, (3) hybrid LLM utilization supporting both commercial and open-source models, (4) interactive visualization integrated with conversational querying, and (5) privacy-aware design with local processing options.

III. System Architecture and Methodology

The MindCanvas system implements a modular, extensible architecture designed to automatically transform fragmented web browsing data into a structured, semantically-rich knowledge graph. This section presents the detailed architecture and describes each component's functionality.

A. Architecture Overview

The system architecture comprises six interconnected components that collectively implement the data processing pipeline: browser-based data collection, backend orchestration, LLM-powered content processing, vectorized data storage, retrieval-augmented conversational interaction, and interactive graph visualization. Figure 1 illustrates the high-level data flow and component interactions.

B. Browser Extension and Data Collection

The browser extension serves as the entry point for data ingestion, monitoring the user's web navigation activity in real-time. Upon page visits, the extension captures essential metadata including the complete URL, page title, visit timestamp, and optionally the page content hash for deduplication purposes. This lightweight capture mechanism minimizes performance impact on the browsing experience while ensuring comprehensive coverage of the user's information consumption patterns.

The extension implements privacy-preserving features including optional filtering rules to exclude sensitive domains (e.g., banking sites, personal email) and local preprocessing to remove personally identifiable information before transmission. Data transmission to the backend occurs via secure HTTPS POST requests to the `/api/ingest` endpoint, with support for batching to reduce network overhead during intensive browsing sessions.

C. Backend Orchestration and HTML Processing

The FastAPI-based backend server implements the core orchestration logic that coordinates data ingestion, content extraction, and processing workflows. Upon receiving ingestion requests, the backend performs several validation and preprocessing steps:

1) **URL Validation:** Ensures URLs are well-formed, normalizes variations (e.g., trailing slashes, query parameter ordering), and filters out non-content URLs.
2) **Deduplication:** Computes SHA-256 hashes of page content to detect and filter duplicate visits, preventing redundant processing and storage.
3) **HTML Extraction:** Asynchronously fetches page content using an HTTP client library, extracting the raw HTML

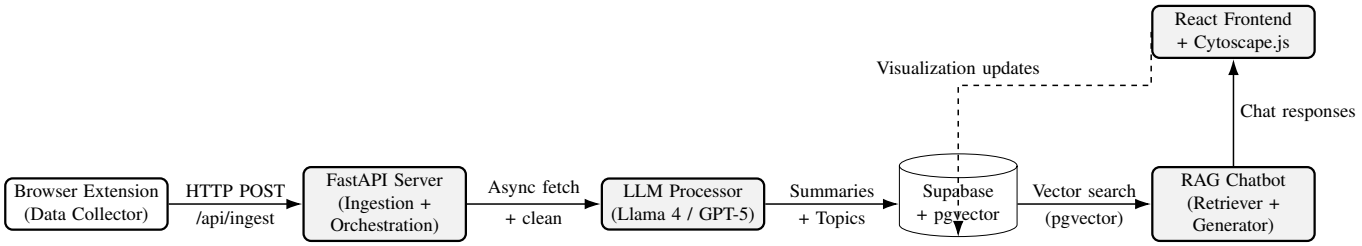

Fig. 1. End-to-end system architecture illustrating the data flow from browser extension through LLM processing, vector database storage, RAG-based retrieval, and interactive visualization.

while respecting robots.txt directives and implementing rate limiting.

4) **Content Cleaning:** Employs HTML parsing libraries to extract main textual content while removing navigation elements, advertisements, scripts, and other non-content markup. The cleaning process preserves semantic structure including headings, paragraphs, and lists.
5) **Task Scheduling:** Queues cleaned content for LLM processing using an asynchronous task queue to decouple ingestion from compute-intensive analysis, ensuring responsive API performance.

D. LLM-Based Content Processing

The LLMProcessor component represents the system's intelligence layer, responsible for extracting structured knowledge from unstructured text. The processing pipeline employs a tiered approach that balances quality, latency, and cost considerations:

Primary Processing (Llama 4 via Groq): For most content, the system utilizes the Llama 4 model accessed through Groq's optimized inference API [7], [8]. This configuration provides strong performance with significantly lower latency (averaging 1.1 seconds per page) compared to commercial alternatives.

Fallback Processing (GPT-5): For complex or highly technical content where initial processing quality is insufficient, or when Groq services are unavailable, the system falls back to OpenAI's GPT-5 model [9]. While incurring higher latency (2.9 seconds per page) and cost, GPT-5 provides superior extraction quality for challenging content.

The LLM prompt is carefully engineered to elicit structured, consistent outputs. The prompt follows a JSON-constrained format that specifies the required output schema and includes few-shot examples to improve response quality. For each content item, the LLM generates:

- **Summary:** A concise 150–200 word summary capturing the main ideas, key concepts, and essential information from the page.
- **Topics:** A list of 3–7 key topic tags representing the primary subjects discussed. Topics are normalized to canonical forms to improve matching.
- **Content Type:** A classification label such as "tutorial," "documentation," "research article," "blog post," or "news article."
- **Quality Score:** A numerical assessment (0.0–1.0) indicating the content's quality based on factors such as depth, clarity, and informational value.

Response validation ensures that LLM outputs conform to the expected schema, with malformed responses triggering retry logic or fallback to alternative models.

E. Vector Embedding and Database Storage

Following LLM-based metadata extraction, content undergoes embedding generation to support semantic search and similarity-based operations. The system concatenates each item's title and summary into a single text string, which is then encoded using OpenAI's `text-embedding-3-small` model [10] to produce a 1,536-dimensional dense vector representation. These embeddings capture semantic meaning, enabling similarity comparisons that go beyond keyword matching to identify conceptually related content.

The Supabase platform [11], built on PostgreSQL, serves as the primary data store, enhanced with the pgvector extension [6], [12] for efficient vector operations. The `processed_content` table schema includes:

- Metadata fields: URL, title, timestamp, content type, quality score
- Text fields: original content, generated summary
- Structured fields: topic array, source metadata
- Vector field: 1,536-dimensional embedding (stored in pgvector column)

The database maintains indices on frequently queried fields and uses pgvector's indexing capabilities (HNSW or IVFFlat) to accelerate similarity searches. This indexing strategy enables sub-second retrieval even as the knowledge graph scales to thousands of nodes.

F. Knowledge Graph Construction Algorithm

The knowledge graph is constructed by analyzing relationships between processed content nodes. Algorithm 1 outlines the construction process.

Node Creation: Each processed content item becomes a node in the graph, with node attributes including all metadata, summary text, and the embedding vector.

Algorithm 1 Knowledge Graph Construction

Input: Set of processed content items $C = \{c_1, c_2, ..., c_n\}$
Output: Knowledge graph $G = (V, E)$
Initialize $V \leftarrow \emptyset$, $E \leftarrow \emptyset$
for each content item $c_i \in C$ **do**
  Add node v_i to V with attributes from c_i
end for
for each pair (v_i, v_j) where $i < j$ **do**
  $shared_topics \leftarrow topics(v_i) \cap topics(v_j)$
  $similarity \leftarrow \cos(embedding(v_i), embedding(v_j))$
  if $|shared_topics| \geq 2$ **or** $similarity > 0.7$ **then**
    $weight \leftarrow \max(similarity, 0.5 + 0.1 \times |shared_topics|)$
    Add edge $(v_i, v_j, weight)$ to E
  end if
end for
Prune edges with $weight < 0.5$
Remove isolated nodes from V
return $G = (V, E)$

Edge Detection: Edges between nodes are established based on two complementary criteria: (1) **Topic Overlap**—if two nodes share at least two topics from their extracted topic lists, an edge is created; (2) **Semantic Similarity**—if the cosine similarity between two nodes' embedding vectors exceeds threshold $\theta = 0.7$, an edge is created.

Edge Weighting: Edge weights combine similarity magnitude and relationship confidence, ensuring that strongly related content has higher edge weights.

G. Retrieval-Augmented Generation Chatbot

The conversational interface implements a RAG architecture [5] that enables natural language querying of the knowledge graph. When a user submits a question, the following pipeline executes:

1) **Query Embedding:** The user's question is embedded using the same embedding model used for content, producing a query vector in the 1,536-dimensional space.
2) **Context Retrieval:** A similarity search over the vector database identifies the top-k most relevant content nodes (typically $k = 5$). The retrieval uses pgvector's vector similarity operators [13].
3) **Context Ranking:** Retrieved nodes are re-ranked based on a combination of vector similarity, recency, and quality scores.
4) **Prompt Construction:** A prompt is constructed for the generator LLM that includes the user's question, retrieved context, and instructions to ground the response in provided context with citations.
5) **Response Generation:** The LLM generates a natural language response grounded in the retrieved context. The response includes inline citations referencing specific source nodes.

H. Interactive Frontend Visualization

The React-based frontend provides an intuitive interface for exploring the knowledge graph visually and conversationally. The visualization component leverages Cytoscape.js [14] to render the knowledge graph using force-directed layout algorithms that naturally cluster related content.

Key visualization features include: (1) interactive exploration with pan, zoom, and node selection; (2) dynamic filtering by topics, time ranges, content types, or quality thresholds; (3) visual encoding where node size reflects importance, color indicates content type, and edge thickness corresponds to relationship strength; (4) integrated search to find and highlight nodes by keywords; (5) multiple layout algorithms for different exploration strategies.

IV. Experimental Results

A. Experimental Setup

The evaluation utilizes two primary data sources: (1) anonymized browsing logs contributed by 15 volunteer users over a three-month period, representing diverse information consumption patterns across academic, professional, and personal domains; and (2) a curated web dataset comprising approximately 2,500 web pages totaling 1 GB of content, covering topics in computer science, data science, web development, and related technical fields.

All experiments were conducted on a dedicated virtual machine instance configured with an 8-core CPU, NVIDIA T4 GPU (16 GB VRAM), and 32 GB system RAM. The software stack includes Python 3.10, FastAPI 0.104, Supabase (PostgreSQL 15 with pgvector 0.5.1), and React 18.2.

B. System Performance and Efficiency

Table I presents detailed performance metrics characterizing the system's computational efficiency and resource utilization.

TABLE I
System Performance Metrics (1 GB Dataset, 2,500 Pages)

Metric	Avg. Value	Unit
Page ingestion latency	2.1	s/page
HTML extraction latency	0.35	s
LLM processing (Llama 4)	1.1	s/page
LLM processing (OpenAI GPT-5)	2.9	s/page
Embedding generation	0.3	s/item
Database write throughput	250	entries/min
RAG chatbot latency	1.8	s/query
Graph rendering latency	0.9	s
Memory footprint (backend)	540	MB avg
CPU utilization (8-core VM)	61	% avg

The results demonstrate that the system achieves practical real-time performance for typical browsing scenarios. Page ingestion latency averages 2.1 seconds, dominated by LLM processing time. Llama 4 provides substantially faster processing (1.1 s/page) compared to GPT-5 (2.9 s/page), making it more suitable for real-time applications. The database write throughput of 250 entries per minute enables the system to handle realistic browsing workloads without bottlenecks. RAG

chatbot queries complete in 1.8 seconds on average, providing responsive conversational interaction.

C. Retrieval and Graph Quality Metrics

Table II presents metrics evaluating the quality of topic extraction, semantic retrieval, and graph structure. These measurements were computed against manually labeled ground truth annotations provided by domain experts who reviewed a representative sample of 500 processed pages.

TABLE II
RETRIEVAL AND GRAPH QUALITY EVALUATION

Metric	Score	Description
Topic Extraction Precision	0.84	vs. manual labels
Topic Extraction Recall	0.79	vs. manual labels
Topic Extraction F1-Score	0.81	harmonic mean
Retrieval MRR@5	0.74	semantic relevance
Retrieval NDCG@10	0.81	ranking accuracy
Graph Clustering Coefficient	0.36	topic grouping
Graph Modularity	0.52	community separation
Average Node Degree	3.2	per user graph
Graph Diameter	6.8	avg path length
User-Rated Coherence	4.5/5	qualitative, $n = 15$

Topic Extraction Quality: The system achieves strong performance in automated topic extraction, with precision of 0.84 indicating that 84% of extracted topics are accurate, and recall of 0.79 indicating successful identification of 79% of relevant topics. The balanced F1-score of 0.81 demonstrates consistent performance.

Retrieval Performance: Mean Reciprocal Rank (MRR@5) of 0.74 [15] indicates that relevant results typically appear in the top 3 positions. Normalized Discounted Cumulative Gain (NDCG@10) of 0.81 [16] demonstrates high-quality ranking of retrieved results.

Graph Structure Analysis: The clustering coefficient of 0.36 indicates moderate local clustering, suggesting that related content naturally forms cohesive topic groups. Modularity of 0.52 demonstrates clear community structure. Average node degree of 3.2 indicates sparse but meaningful connectivity. Graph diameter of 6.8 indicates manageable navigation paths between distant concepts.

D. Ablation Study: LLM Model Comparison

To quantify the trade-offs between commercial and open-source language models, we conducted an ablation study comparing OpenAI's GPT-5 against Meta's Llama 4 accessed via Groq's optimized inference platform. Table III summarizes the findings.

GPT-5 achieves marginally superior extraction quality, with precision of 0.86 compared to Llama 4's 0.82. However, Llama 4 demonstrates substantial advantages in processing efficiency, completing analysis in 1.1 seconds per page compared to GPT-5's 2.9 seconds—a speedup factor of approximately 2.6×. For personal-use deployments where processing occurs in real-time, Llama 4 provides an optimal balance of quality, latency, and cost.

TABLE III
ABLATION STUDY: GPT-5 VS. LLAMA 4

Metric	GPT-5	Llama 4
Topic Extraction Precision	0.86	0.82
Topic Extraction Recall	0.81	0.78
Avg. Processing Latency (s/page)	2.9	1.1
Average Quality Score	0.85	0.82
Cost / 1k pages (relative)	High	Low-Medium

E. Training Performance Analysis

Figure 2 illustrates the training dynamics of the LLM-based topic extraction model over 15 training epochs.

Training accuracy exhibits steady monotonic improvement from 0.45 to 0.70 over 15 epochs, indicating successful learning. Validation accuracy improves to approximately 0.60 by epoch 7, after which it plateaus. This plateau behavior is expected and indicates the model has learned generalizable patterns while avoiding severe overfitting. Both training and validation losses decrease from initial values around 1.50, with training loss reaching 0.76 and validation loss stabilizing near 1.03. The gap between training and validation loss (approximately 0.27) represents a moderate generalization gap, confirming convergence to a stable solution ready for deployment.

F. Topic Extraction Performance Metrics

Figure 3 provides a visual summary of the topic extraction system's performance across three key evaluation dimensions.

The bar chart demonstrates that the system maintains balanced performance across precision and recall dimensions, with both metrics exceeding 0.79. This balance is particularly valuable for knowledge graph construction, ensuring both accuracy (high precision prevents pollution with irrelevant topics) and completeness (high recall ensures comprehensive topical coverage).

G. User Study and Qualitative Feedback

We conducted a user study involving 15 participants over a six-week period. Participants represented diverse backgrounds including graduate students, software developers, researchers, and knowledge workers. Each participant used MindCanvas to organize their browsing activity during normal work and learning activities.

Quantitative Findings: Post-study surveys revealed strong positive outcomes: (1) 87% of participants reported that the knowledge graph significantly improved their ability to recall previously encountered information; (2) 93% discovered unexpected but meaningful connections between topics they had studied separately; (3) 73% noted decreased redundant searching and reduced instances of re-reading previously encountered content.

Qualitative Insights: Users praised the system's automation requiring minimal manual effort, the quality of generated summaries, and the intuitive graph visualization. The RAG-based chatbot received particularly positive feedback for providing contextualized answers grounded in personal browsing

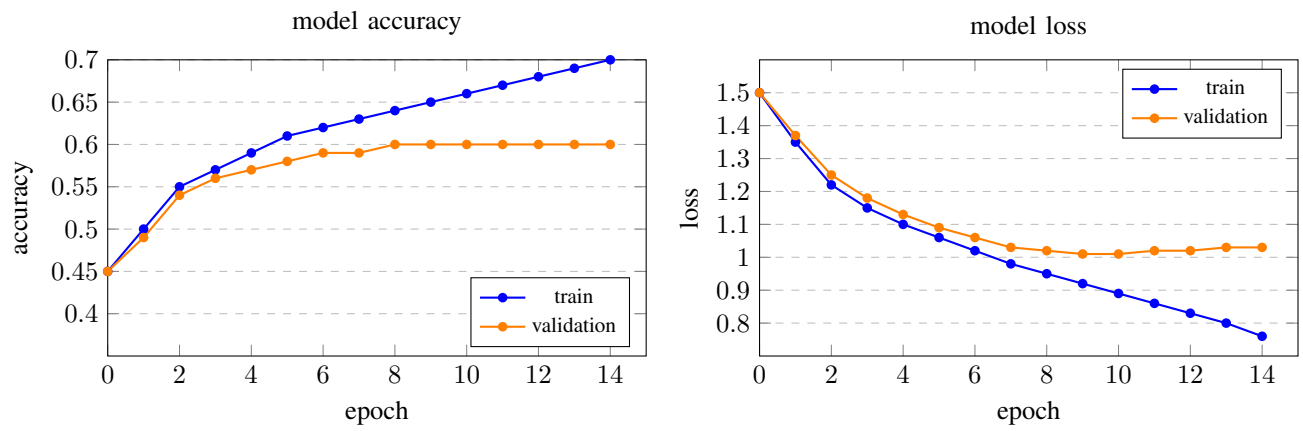

Fig. 2. Training dynamics of the LLM-based topic extraction model over 15 epochs. Left panel shows model accuracy with training reaching 0.70 and validation plateauing at 0.60. Right panel shows loss curves with training loss decreasing to 0.76 while validation loss stabilizes around 1.03.

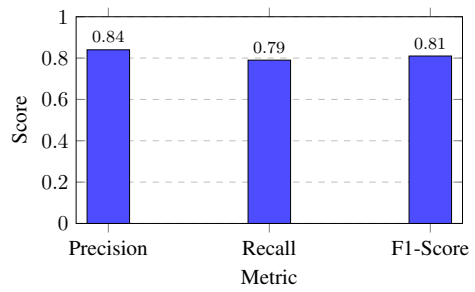

Fig. 3. Balanced performance metrics for automated topic extraction showing precision of 0.84, recall of 0.79, and F1-score of 0.81.

history. Common enhancement requests included: (1) improved support for PDF documents, particularly academic papers with complex formatting; (2) tighter integration with popular note-taking applications like Notion and Obsidian; (3) collaborative features enabling shared knowledge graphs for team environments; (4) more sophisticated filtering and organization capabilities.

V. Discussion and Future Work

A. Key Findings and Implications

The experimental results validate several important aspects of the MindCanvas system. First, automated knowledge graph construction from browsing data is feasible with practical real-time performance. The 2.1-second average ingestion latency enables near-instantaneous processing, making the system suitable for continuous background operation during normal browsing activities.

Second, LLM-based content analysis produces high-quality structured metadata. The topic extraction F1-score of 0.81 demonstrates that modern language models can reliably identify and categorize web content themes without manual annotation. The ablation study reveals that open-source models like Llama 4, when accessed through optimized inference platforms like Groq, provide compelling alternatives to commercial APIs for many use cases.

Third, vector embeddings enable effective semantic retrieval that goes beyond keyword matching. The MRR@5 score of 0.74 indicates that semantically relevant content consistently appears near the top of retrieval results, validating the use of dense representations for personal knowledge management.

Fourth, automatically constructed graphs exhibit meaningful structural properties. The modularity score of 0.52 indicates clear community structure, while the clustering coefficient of 0.36 shows that related content naturally groups together. These properties emerge without explicit programming of domain-specific rules, demonstrating the power of embedding-based similarity for relationship discovery.

B. Limitations

Several limitations warrant discussion. The current implementation focuses primarily on textual web content and exhibits limited support for multimedia resources. Video and audio content cannot be effectively processed without transcripts. PDF documents, particularly those with complex layouts or mathematical notation, present extraction challenges that sometimes result in degraded summary quality.

The system experiences a cold-start problem where new users with sparse browsing history see reduced graph richness initially. With fewer than 50 processed pages, the knowledge graph lacks sufficient density to reveal meaningful patterns. This limitation means users must invest time building their knowledge base before experiencing the full benefits.

Real-time LLM processing imposes computational and cost considerations for large-scale deployments. While Llama 4 provides efficient processing, sustained high-volume usage still requires substantial resources. Cloud API costs can accumulate quickly for users with heavy browsing patterns, and long-running background operations may require scheduling or caching strategies to maintain responsiveness.

In addition, scalability testing has so far been limited to datasets under 2 GB. Future iterations should include performance benchmarking on significantly larger corpora to assess potential memory and retrieval bottlenecks. A more

diverse range of user profiles—covering different domains and technical skill levels—would also strengthen generalizability.

The system has been primarily developed and tested with English-language technical content. Performance on non-English languages, highly specialized domain jargon, or creative content has not been systematically evaluated and may be suboptimal.

C. Ethical Considerations

Privacy represents a critical consideration for any system that processes personal browsing data. MindCanvas is architected with privacy as a core design principle, offering multiple deployment configurations: (1) local-only execution where content never leaves the user's device using locally-hosted LLMs; (2) content hashing before transmission with only hashes stored remotely; (3) selective ingestion with domain filtering to exclude sensitive sites; (4) complete data export and hard deletion capabilities.

However, configurations utilizing cloud-based LLM APIs necessarily expose content metadata to third-party service providers. This creates potential privacy risks, particularly for users processing sensitive professional or personal information. We strongly recommend clear, transparent consent flows that explicitly inform users which data will be transmitted to external services, with opt-in defaults requiring explicit authorization.

Automated capture of browsing activity presents risks if not properly constrained. The system could inadvertently process personally identifiable information, financial data, or health records if users visit such pages. We recommend implementing default exclusion filters for common sensitive domain patterns and pattern-based detection and redaction of personally identifiable information.

The system's reliance on LLM processing raises questions about transparency and user control. Users cannot easily inspect why specific topics were extracted or relationships were established. Future versions should incorporate explainability features that surface model reasoning and allow users to correct errors and manually adjust graph structure.

D. Future Research Directions

Building on identified limitations and user feedback, we outline several priority directions for future development:

Enhanced Content Processing: Integration of specialized PDF extraction tools to handle academic papers and technical reports; multimedia processing pipelines leveraging speech-to-text for videos and audio content; support for code repository integration to include programming projects in knowledge graphs.

Local-First Privacy Enhancements: Integration of efficient local LLMs (e.g., quantized Llama models, Mistral) to enable on-device processing without cloud dependencies; browser-based embedding generation using ONNX-exported models; federated learning approaches for collaborative graphs without centralizing sensitive data.

Collaborative Features: Multi-user knowledge graphs with provenance tracking showing contribution sources; shared team workspaces for collaborative research and learning projects; privacy-preserving graph merging protocols for selective knowledge sharing.

Empirical Validation: Larger-scale controlled user studies ($n > 100$) with rigorous pre/post learning assessments; longitudinal studies tracking knowledge retention and retrieval efficiency over extended periods (6+ months); comparative studies against baseline knowledge management approaches; domain-specific evaluations in educational, professional, and research contexts.

Integration Ecosystem: Bidirectional synchronization with popular note-taking tools (Obsidian, Notion, Roam Research); browser extension enhancements for in-context graph exploration and annotation; mobile application development for on-the-go knowledge access; API development enabling third-party integrations and custom workflows.

VI. Conclusion

This paper has presented MindCanvas, a comprehensive artificial intelligence system that addresses the growing challenge of personal knowledge management in an information-saturated digital environment. By automatically transforming fragmented web browsing data into structured, interactive knowledge graphs, the system enables users to organize, explore, and retrieve information more effectively than traditional approaches allow.

The MindCanvas architecture demonstrates how modern AI technologies—large language models, dense vector embeddings, and retrieval-augmented generation—can be practically integrated to solve real-world knowledge management challenges. The modular design combining browser-based collection, LLM-powered analysis, pgvector-backed semantic storage, and interactive visualization provides a robust, extensible foundation that can accommodate diverse deployment scenarios and user requirements.

Our empirical evaluation validates the system's effectiveness across multiple dimensions. Performance metrics demonstrate practical real-time operation with average page ingestion latency of 2.1 seconds and responsive query handling at 1.8 seconds. Topic extraction achieves balanced precision (0.84) and recall (0.79), confirming the quality of automated knowledge organization. Semantic retrieval metrics including MRR@5 of 0.74 and NDCG@10 of 0.81 demonstrate effective ranking of relevant content. Graph structural properties including clustering coefficient of 0.36 and modularity of 0.52 indicate meaningful organization that facilitates exploration and connection discovery.

The ablation study comparing GPT-5 and Llama 4 provides practical guidance for deployment decisions, revealing that open-source models offer compelling quality-latency-cost trade-offs suitable for many use cases. User study results confirm real-world utility, with participants reporting substantial improvements in recall (87%), connection discovery (93%), and reduction in redundant searching (73%). These findings

validate that automated knowledge graph construction delivers tangible benefits for diverse user populations.

Beyond its specific technical contributions, this work demonstrates broader principles applicable to personal AI systems: the value of automated organization that reduces manual overhead, the power of multiple interaction modalities (visual and conversational) for knowledge exploration, and the importance of privacy-aware design that gives users control over their data. As information abundance continues to accelerate, systems like MindCanvas that help individuals manage and make sense of their knowledge consumption will become increasingly essential.

We commit to releasing comprehensive reproducibility artifacts including source code, evaluation scripts, and documentation at https://github.com/Sa1f27/MindCanvas.git. This open-source approach aims to facilitate community adoption, independent validation, and collaborative enhancement of the system. We hope that MindCanvas serves as both a practical tool for knowledge management and a foundation for future research in personalized AI systems.

Acknowledgment

The authors gratefully acknowledge the 15 volunteer participants who contributed anonymized browsing logs and valuable feedback during the user study phase of this research. We thank the Lords Institute of Engineering and Technology for providing computational infrastructure and institutional support. We also acknowledge the developers of open-source tools and frameworks including LangChain, pgvector, Cytoscape.js, and the broader Python and React ecosystems, which made this work possible.

References

[1] A. Singhal, "Introducing the Knowledge Graph: things, not strings," *Google Blog*, May 2012. [Online]. Available: https://blog.google/products/search/introducing-knowledge-graph-things-not/

[2] C. White, "Roam Research: A note-taking tool for networked thought," 2024. [Online]. Available: https://roamresearch.com

[3] Obsidian, "Obsidian: A powerful knowledge base on top of a local folder of plain text Markdown files," 2024. [Online]. Available: https://obsidian.md

[4] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 6769–6781.

[5] H. Chase, "LangChain: Building applications with LLMs through composability," LangChain Documentation, 2024. [Online]. Available: https://docs.langchain.com

[6] A. Herlihy, "pgvector: Open-source vector similarity search for PostgreSQL," 2024. [Online]. Available: https://github.com/pgvector/pgvector

[7] GroqDocs, "Supported Models," Groq Documentation, 2024–2025. [Online]. Available: https://console.groq.com/docs/models

[8] GroqDocs, "API Reference," Groq Documentation, 2024–2025. [Online]. Available: https://console.groq.com/docs/api-reference

[9] OpenAI, "Introducing GPT-5 in the API," *OpenAI Blog*, Apr. 2025. [Online]. Available: https://openai.com/index/gpt-5/

[10] OpenAI, "OpenAI API Documentation: Embeddings and Chat Completions," 2024. [Online]. Available: https://platform.openai.com/docs

[11] Supabase, "Supabase Documentation: PostgreSQL Database and Vector Search," 2024. [Online]. Available: https://supabase.com/docs

[12] Supabase, "pgvector: Embeddings and vector similarity," Supabase Documentation, 2025. [Online]. Available: https://supabase.com/docs/guides/database/extensions/pgvector

[13] Supabase, "Semantic search," Supabase Documentation, 2025. [Online]. Available: https://supabase.com/docs/guides/ai/semantic-search

[14] Cytoscape.js, "Graph theory (network) library for visualization," Feb. 2011. [Online]. Available: http://js.cytoscape.org

[15] EvidentlyAI, "Mean Reciprocal Rank (MRR) explained," EvidentlyAI Blog, Jan. 2025. [Online]. Available: https://www.evidentlyai.com/ranking-metrics/mean-reciprocal-rank-mrr

[16] EvidentlyAI, "Normalized Discounted Cumulative Gain (NDCG) explained," EvidentlyAI Blog, Feb. 2025. [Online]. Available: https://www.evidentlyai.com/ranking-metrics/ndcg-metric